

SQL-2026

(password): Shivam@SQL2026

< Starting-SQL >

#SELECT : Use to select actual table that we are going to querying off.

(*) : It means everything (ie table, row, etc)

#FROM : It specifying what table.

Example :

Important note:

```
SELECT *  
FROM exp-table;
```

(;) : tells this is the end of this query.

Defining Databases from where table belongs →

```
SELECT *  
FROM exp-database.exp-table;
```

Select one Column →

```
SELECT exp-column  
FROM exp-database.exp-table;
```

Select multiple Column →

```
SELECT exp-column1, exp-column2, exp-column3  
FROM exp-database.exp-table;
```

Operation on some Column →

Important note:

{ i.e. age →
age + 10 }

```
SELECT exp-column1,  
exp-column1 (+ operation)  
FROM exp-database.exp-table;
```

Find Only or Select only distinct element →

```
SELECT DISTINCT exp-column  
FROM exp-database.exp-table;
```

-- WHERE Clause →

WHERE : Used to help filter our records [we only going to return the rows that fulfill a specific condition]

```
WHERE exp-column = 'xyz' / 123  
;
```

Comparison operator : != → Not equals to

Important note: = → equals to

>, < → greater than / less than

>= → greater or equals to

<= → smaller / less or equals to

-- Logical operators

(u) **AND** - on additional conditional statement to add
↳ union

(n) **OR** - either of condition is true for output
↳ intersection

NOT -

We use parentheses to met the both condition

[Condition] → isolated condition

-- Like Statement [sequence match]

% : It means anything (before or after)

_ : It means a specific value

Important note:

SELECT *

FROM exp-table

WHERE exp-column1 **LIKE** 'x%', '%x%', '%x'

WHERE exp-column1 **LIKE** 'x-', 'x_', 'x--%'

Date:

M T W T F S S

-- Group By

It group together rows that have the same value in the specified column.

GROUP BY exp-column

→ use for aggregate function [AVG, SUM, MIN, MAX, COUNT]

Exp:

```
SELECT exp-column1, AVG(exp-column2)
FROM exp-table
GROUP BY exp-column1
;
```

-- Order By

Important note: → sort the result set in either ascending or descending order

{ ASC : A-Z, 1-∞
DESC : Z-A, ∞-1 }

Date:

Exp:

```
SELECT *  
FROM exp-table  
ORDER BY Column 1, Column 2  
;
```

-- HAVING →

It work to sort or filter only aggregated function.

HAVING (aggregate fⁿ) = value

Exp:

```
SELECT * Avg(salary)  
FROM exp-column  
HAVING AVG(salary) > value.
```

Important note:

-- LIMIT & Aliasing

How many rows you want in your Output - limit

Date:

Exp:

```
SELECT *  
FROM exp-column / exp-table  
LIMIT value.
```

Exp

```
SELECT *  
FROM exp-table  
LIMIT 2, 1  
      ↑  
      initial (limit size)
```

Use to change the name - Aliasing

```
SELECT Column 1, Avg(Column 2) AS Avg-age  
FROM exp-table  
GROUP BY Column 1  
HAVING Avg-age > 00;
```

Important note:

Date: _____

-- INSERT, UPDATE & DELETE:

INSERT INTO table-name (column 1, — column n)
VALUE = (value 1, — — value n);

UPDATE table name
SET column 1 = value
column 2 = value

WHERE condition;

DELETE FROM table-name
WHERE condition;

Summary :

Select statement used to fetch data

Where used to extract only those specific records those fulfill condition

Distinct ⁿ specify unique values in specified column

Limit is a clause that specify the maximum no. of rows.

Insert is used to insert rows in table

Delete is used to delete rows in table

Important note: **Update** is used to update the data in table